Naive and Tiled Matrix Multiplication

Course: Massively Parallel Programming on GPUs

Teacher: Donato D'Ambrosio
Master Degree in Artificial Intelligence and Computer Science
Department of Mathematics and Computer Science
University of Calabria, Italy

November 4, 2026

1 Introduction

This CUDA lab consists of:

- Developing a CPU baseline matrix multiplication implementation in C/C++ and OpenMP, for comparison;
- Developing a naive CUDA matrix multiplication kernel using global memory only;
- Developing a tiled CUDA matrix multiplication kernel using global memory and shared memory;
- Developing a coarse-grained tiled CUDA matrix multiplication kernel using global memory and shared memory;
- Checking correctness, assessing performance, and analysing the experimental results.

2 Matrix Multiplication

Consider the following matrices and their dimansions:

$$A \in \mathbb{R}^{M \times K}, \qquad B \in \mathbb{R}^{K \times N}, \qquad C \in \mathbb{R}^{M \times N}.$$

The matrix multiplication, $C = AB$, is defined such that each output element is computed as:

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}, \qquad 0 \leq i < M, \quad 0 \leq j < N.$$

Each implementation must use single-precision floating point values and support arbitrary valid values of M, N, and K.

Initialize the input matrices using the following values:

- All the elements of A may be set to $0.5 \cdot 2^{-10}$;
- All the elements of B to 2;
- All the elements of C to 0.

3 OpenMP Implementation

Implement a baseline CPU version of matrix multiplication in C/C++ and OpenMP. The implementation must define the `checkCorrectenss()` function to check result's correctness.

4 CUDA Implementations

Implement the following two CUDA versions of matrix multiplication.

4.1 Naive Matrix Multiplication (Global Memory only)

Assign one CUDA thread to each output matrix element C_{ij}. Each thread computes the corresponding dot product by iterating over the K elements of the input matrices.

The kernel must:

- Check the output indices against the matrix dimensions.
- Read the required elements of A and B from global memory;
- Compute the dot product for one output element;
- Write the result to global memory;

Check for the correctness by reusing the host-side `checkCorrectenss()` function.

Use a two-dimensional grid and two-dimensional thread blocks. The grid dimensions must cover all elements of C, including cases in which M or N is not divisible by the block dimensions.

4.2 Tiled Matrix Multiplication (Global and Shared Memory)

Implement a tiled CUDA matrix multiplication kernel using global memory and shared memory.

Divide the output matrix C into square tiles of dimensions $T \times T$. Each CUDA block computes one output tile, and each thread computes one element of that tile.

Use two shared-memory arrays to store the corresponding tiles of A and B.

For each tile along the reduction dimension K:

- Cooperatively load a tile of A from global memory into shared memory;
- Cooperatively load a tile of B from global memory into shared memory;
- Synchronize the threads in the block;
- Compute partial dot products using the shared-memory tiles, and accumulate it into a register;
- Synchronize the threads before loading the next pair of tiles.
- Repeat until the element's dot product is complete, and write its final result to global memory.

Check for the correctness by reusing the host-side `checkCorrectenss()` function.

The implementation must handle matrix dimensions that are not multiples of T. Out-of-range elements loaded into shared memory must be set to zero. Threads whose output indices are outside the dimensions of C must not write to the output matrix.

4.3 Coarse-Grained Tiled Matrix Multiplication

Based on the Tiled Matrix Multiplication kernel, implement a coarse-grained tiled CUDA matrix multiplication kernel using global memory and shared memory.

Each block computes n_t tiles of the target matrix along the reduction dimension K. The kernel:

- Cooperatively load a tile of A from global memory into shared memory;
- Cooperatively load n_t tiles of B from global memory into shared memory;
- Synchronize the threads in the block;
- Compute the n_t partial dot products using the shared-memory tiles, and accumulate them into n_t registers;
- Synchronize the threads before loading the next $(n_t + 1)$ tiles.
- Repeat until the elements' dot products are complete, and write their final results to global memory.

Check for the correctness by reusing the host-side `checkCorrectenss()` function.

The implementation must handle matrix dimensions that are not multiples of T. Out-of-range elements loaded into shared memory must be set to zero. Threads whose output indices are outside the dimensions of C must not write to the output matrix.

5 Performance Assessment, Correctness, and Analysis

Evaluate all implementations using the same input matrices.

5.1 Matrix Dimensions

For comparison, use the following matrix dimensions:

$$(M, K, N) = (2^{11}, 2^{10}, 2^9)$$

Preliminary verify if the three matrices fit comfortably within the available GPU memory[1]. If this is not the case, iteratively reduce the common dimension of a factor 2 until the three matrices fit the GPU memory.

5.2 Block and Tile Configurations

Use the following block configurations:

$$8 \times 8$$
$$16 \times 16$$
$$32 \times 32$$

Accordingly, for the tiled implementations, use the corresponding tile dimensions:

$$T \in \{8, 16, 32\}.$$

The shared-memory allocation for the tiled implementation must be sufficient for two $T \times T$ tiles of single-precision floating-point elements[2]:

[1] The NVIDIA GTX 980 installed on the JPDM2 workstation has 4043 MiB of device memory.
[2] The NVIDIA GTX 980 installed on the JPDM2 workstation has 96 KiB of shared memory.

$$S_{\text{block}} = 2 \times T^2 \times 4 \quad \text{bytes.}$$

The shared-memory allocation for the coarse-grained tiled implementation must be sufficient for $(n_t + 1)$ $T \times T$ tiles of single-precision floating-point elements:

$$S_{\text{block}} = (n_t + 1) \times T^2 \times 4 \quad \text{bytes.}$$

5.3 Correctness

Compare every CUDA output matrix against the CPU OpenMP reference.

5.4 Performance Assessment

Performance assessment must be done for the matrix multiplication kernel only.

Each performance assessment experiment must execute:

- One warm-up run;
- Four additional runs (repetitions);

Compute and report the average execution time of the four repetitions. I suggest using a script for this purpose.

For assess the performance of the OpenMP implementation of the matrix multiplication kernel on the JPDM2 workstation use all eight CPU cores.

5.5 Performance Analysis

Produce tables and plots comparing the CPU and GPU implementations.

Include:

- A bar plot for each CUDA implementation reporting the execution times for each block configuration;
- A bar plot comparing the execution times of all implementations, including those of the CPU implementation. For the CUDA implementations, choose the minimum elapsed time among those obtained by varying the block configuration.